

THE ENGINEERING BAR FOR THE AI LAYER

WEEK 1, DAY 3

Seam, boundary, layers. Foundation layer.





THE ASSUMPTION NOBODY SAID OUT LOUD

TWO DAYS OF GATES, ONE HIDDEN PREMISE

BOTH GATES ASSUME THE CODE DOES THE SAME THING TWICE.

ruff never runs it. mypy reasons about your code, not the reply.
pytest is the one that breaks.

THE SPINE OF THE DAY

SEAM, BOUNDARY, LAYERS

IN THAT ORDER. EACH ONE NEEDS THE ONE BEFORE IT

1 seam

So your tests do not call the model at all.

2 boundary

So the type checker does not go blind on an untyped reply.

3 layers

Two suites, asking two different questions.

The bar does not drop for the AI layer. The scaffolding changes.

THE TEST EVERYBODY WRITES FIRST

AND IT IS WRONG

THE NAIVE TEST

```
def test_extracts_the_amount():  
    assert extract_amount("total is 35.35") == "35.35"
```

WHAT THE MODEL ACTUALLY RETURNS

```
PHRASINGS = (  
    '{"amount": "35.35"}',  
    'Sure! Here you go:\n{"amount": "35.35"}',  
    '```json\n{"amount": "35.35"}\n```',  
    '{"amount": 35.35}',  
    'I could not find an amount in that line.',  
)
```

Four of the five contain the right answer. The model is not malfunctioning.

FIVE PASSES IN TWENTY

VERIFIED OVER TWENTY RUNS

TWENTY CONSECUTIVE RUNS

```
FFFFFF.FF...FFFF.FFF
```

```
passed: 5   failed: 15   of 20
```

NOTE

The exact number is random.
Quote your own run.

A test that always fails is a nuisance. A test that fails one run in four is a liability, because the team learns to re-run it.

THREE PROBLEMS, THREE FIXES

ALL THREE ARE IN THAT ONE FUNCTION

WHAT IS ACTUALLY WRONG

- 1 seam** It builds its own client, so there is no way to test it without a model.
- 2 boundary** json.loads hands back an untyped blob and we index into it and hope.
- 3 layers** Our one test asserts on the model's prose.

Seam, boundary, layers. In that order, because each needs the one before it.

THE SEAM: MAKE THE CLIENT AN ARGUMENT

ONE WORD OF CHANGE

BEFORE

```
def extract_amount(line):  
    client = FakeModel()
```

AFTER

```
def extract_amount(  
    line: str,  
    client: ModelClient,  
) -> Decimal:
```

The function decided for itself what it talked to. So there was no point in the program where you could hand it something else.

TWO IMPLEMENTATIONS, ONE PROTOCOL

STRUCTURAL TYPING

THE SEAM ITSELF

```
class ModelClient(Protocol):  
    def complete(self, prompt: str) -> str: ...
```

FAKEMODEL

Non-deterministic.
Stands in for production.

STUBMODEL

Returns exactly what you
constructed it with.

Structural, so the provider's SDK satisfies it without inheriting anything of yours.

TWO FUNCTIONS, BOTH ANNOTATED

PREDICT THE ANSWER BEFORE I RUN IT

BOUNDARY_UNTYPED.PY

```
def amount_from_response(payload: str) -> Decimal:
    data = json.loads(payload)
    return data["amount"]
```

```
def doubled(payload: str) -> Decimal:
    data = json.loads(payload)
    return data["amount"] * 2
```

What does mypy say?



MYPY, WITH A DEFAULT CONFIGURATION

ON THE CODE YOU JUST READ

**SUCCESS: NO ISSUES
FOUND IN 1 SOURCE FILE**

Not a warning. Not a note. Success.

ANY IS NOT UNKNOWN

WHY THE GATE WENT SILENT

Any is a formal instruction to the type checker to stop checking.

FOUR STEPS TO A SILENT GATE

- `json.loads` is annotated as returning `Any`.
- Every expression derived from an `Any` is `Any`.
- `Any` is compatible with every type, in both directions.
- So nothing downstream can ever be an error.

You did not write `Any`. The standard library handed it to you at a boundary, and it spread.

WHAT STRICT MODE DID NOT SAY

IT SEES THE ESCAPE, NOT THE ERROR

MYPY --STRICT

```
error: Returning Any from function  
declared to return "Decimal" [no-any-return]
```

It never mentioned `data["amount"] * 2`

AND AT RUNTIME

```
amount_from_response -> '35.35'      type: str  
doubled               -> '35.3535.35'
```

String repetition. It did not crash. No gate objected to the arithmetic.



THE SENTENCE TO TAKE AWAY

IF YOU KEEP ONE THING FROM TODAY

**PARSING TURNS TEXT INTO A BLOB.
VALIDATING TURNS A BLOB
INTO A TYPE.**

Only the second one lets a type checker help you.

TWO FAILURES, ANSWERED DIFFERENTLY

NAME THEM SEPARATELY

MODELREFUSED

No JSON at all. A legitimate outcome of asking a model a question.

Often transient. Retry.

MODELCONTRACTVIOLATION

JSON present, schema broken. A float where a string was required.

Not transient. Do not retry.

Retrying a contract violation is a hundred times the cost for the same failure.

TWO LAYERS, TWO QUESTIONS

UNIT AND CONTRACT

THEY ASK DIFFERENT THINGS

1 unit

Given THIS reply, does our code behave? Stubbed. Every commit.

2 contract

Whatever the model says, does our code hold? Calls it. Nightly.

HOW YOU SEPARATE THEM

```
pytest -m "not contract"    # the fast gate
pytest -m contract          # nightly
```

Nearly all your tests live in the first layer, and none of them call a model.



THE CONTRACT-TEST INSIGHT

WHAT YOU CAN AND CANNOT ASSERT

**YOU CANNOT ASSERT AN OUTPUT.
YOU CAN ASSERT THAT EVERY
OUTPUT IS HANDLED.**

Two hundred calls, and every reply landed in a case you accounted for.

FIVE IN TWENTY BECOMES TWENTY IN TWENTY

SAME MODEL, DIFFERENT SCAFFOLDING

5/20

THIS MORNING, NAIVE TEST

20/20

BOTH LAYERS, AFTER

Nothing about the model changed. Same class, same five random strings. The bar did not drop; the scaffolding changed.

And one of those layers is calling the non-deterministic thing two hundred times per run.

PRACTICAL: RUFF SAYS ALL CHECKS PASSED

FORTY-FIVE MINUTES, ALONE

THE THREE GATES ON AN UNSEEN MODULE

```
$ ruff check .  
All checks passed!
```

```
$ mypy src tests      -> 7 errors, ONE of them AI-specific  
$ pytest -q (20 runs) -> passed 17 / failed 3
```

Seven defects. Three of them MACHINE. 85 percent reads as green, so the team learns to re-run it.

AI usage: restricted from here. Gate, then yesterday's checklist, then four new questions.

THE PASS BAR

ALL SIX

- 1 The Returning Any finding, classified MACHINE, mypy line referenced
- 2 The unvalidated category, classified HUMAN, with runtime evidence
- 3 The missing seam named as why the module cannot be tested
- 4 The retry loop named as retrying a non-retryable failure
- 5 Flakiness identified WITH THE RATE STATED
- 6 Verdict of request changes

"Sometimes fails" is not a finding. A rate is.

NOT ONE MINUTE ON PROMPTS TODAY

A brilliant prompt behind an untested seam
is still an untested seam.

Prompt quality is real, and it is Week 2. Today was the engineering around the call.

TOMORROW: SPEC-DRIVEN DEVELOPMENT. THEN FRIDAY: THE WHOLE LOOP.